PROJECT ARCHITECTURE | FINAL TECH STACK v3.0

CISGuard 360

Unified CIS Compliance, Cross-Layer Security Intelligence & Proof-of-Hardening Platform

38	30	20	88
Mandatory PS	Industry extras	Innovations	Total features

Core foundation: React + TypeScript, Python + FastAPI, Supabase (PostgreSQL, Auth, Storage, Realtime), Celery + Redis, OpenSCAP + ComplianceAsCode, custom CIS rule engine, NetworkX, OR-Tools, Batfish, Ansible, OpenBao and a full testing toolchain.

Design constraints: Full original PS preserved; no runtime LLM / AI-ML; no paid third-party API dependency; actual device evidence; approval-gated remediation and honest reporting of unsupported or unverified checks.

Prepared 29 October 2026 | Working product name | Architecture and development reference

Status This is the **final proposed technology selection**, not a claim of completed implementation. All 88 feature entries remain in scope; correct operation must be established through integration and validation tests.

1. Executive architecture decisions

Area	Selected technology	Reason
Frontend	React + TypeScript + Vite; Tailwind CSS; shadcn/ui	Modular dashboard and responsive UI
Backend	Python + FastAPI; Pydantic; SQLAlchemy	APIs, rule logic and tool integration
Database / Auth	Supabase PostgreSQL + Supabase Auth + MFA + RLS	Unified application identity and audit data
Storage / Realtime	Supabase Storage + Realtime	Protected artifacts and live dashboard progress
Workers	Celery + Redis + Celery Beat + Nornir	Multi-device scheduled and parallel execution
OS compliance	OpenSCAP + ComplianceAsCode + osquery + Windows tooling	Standards support and system facts
Device collection	Netmiko, Scrapli, WinRM, SSH, vendor APIs	Authorized live configurations
Rule evaluation	Custom deterministic Python + YAML/JSON/XML	Versioned, explainable CIS rules
Security intelligence	NetworkX + custom graph algorithms	Cross-device relationships and risk correlation
Remediation optimization	Google OR-Tools CP-SAT	Constraint-based minimal-change plans
Simulation	Batfish + Pybatfish + custom models	Supported network impact analysis
Hardening	Ansible + Ansible Runner + PowerShell + vendor APIs	Approved controlled changes
Evidence / reports	Cryptography, Jinja2, WeasyPrint	Verifiable artifacts and exports
Secrets	OpenBao	Separate privileged device credential vault
Deployment / observability	Docker Compose; Linux; Caddy/Nginx; Prometheus/Grafana/Loki	Portable deployment and operations

2. What Supabase replaces - and what remains

Previously proposed	Final choice	Treatment
Standalone PostgreSQL	Supabase PostgreSQL	Replace standalone database deployment
Keycloak	Supabase Auth	Replace for application user authentication
Separate file storage	Supabase Storage	Private encrypted snapshots and reporting artifacts
Custom realtime infrastructure	Supabase Realtime + FastAPI WebSockets	Hybrid: dashboards plus job/control channels
Device credential secrets	OpenBao	Retain separate dedicated secrets manager
Celery / Redis	Celery / Redis	Keep for orchestration
ORM / migrations	SQLAlchemy + Alembic	Keep; coordinate with Supabase CLI migrations

Critical security boundary Supabase Auth identifies users. FastAPI still verifies tokens, enforces organization-scoped authorization and controls privileged device operations. Browser clients never receive firewall secrets, service-role keys or unrestricted infrastructure access.

3. Frontend libraries and developer tools

Library / tool	Use
react; typescript; vite	Main SPA, type checking and build tooling
tailwindcss; shadcn/ui	Styling, accessible dashboard forms/dialogs
lucide-react	Icons
react-router-dom	Routing and pages
@tanstack/react-query	API querying, caching and synchronization
@tanstack/react-table	Large device, findings and audit tables
react-hook-form; zod	Validated forms and schemas
recharts	Compliance, severity and audit-trend charts
cytoscape; react-cytoscapejs	Cross-device risk graph visualizations
@xyflow/react	Remediation dependency and workflow graphs
@supabase/supabase-js	Auth, session handling, Storage and Realtime
date-fns; sonner	Time/date display, notifications
vitest; @testing-library/react	Frontend tests
VS Code / Antigravity IDE; Node.js; pnpm/npm; Chrome DevTools; Figma (optional)	Development environment and UI design

4. Python backend - complete libraries

Category	Libraries / frameworks	Role
API / application	fastapi, uvicorn	REST APIs, WebSockets
Validation	pydantic, pydantic-settings	Typed input/output and runtime configuration
Supabase	supabase (Python SDK)	Auth, Storage and other integrations
Database	sqlalchemy, psycopg, alembic	SQL, PostgreSQL drivers, migrations
JWT and crypto	PyJWT, cryptography, httpx	JWT/JWKS verification, encryption, HTTP
Jobs	celery, redis	Background jobs and queue broker
Rule formats	pyyaml, jsonschema, lxml	YAML/JSON/XML and schema validation
Config parsing	ttp, textfsm, ciscoconfparse2	Structured vendor CLI configuration parsing
SSH / transport	netmiko, scrapli, paramiko	Network connections and command retrieval
Windows remote	pywinrm, pypsrp	WinRM and PowerShell remoting
Network orchestration	nornir	Network inventory and task dispatch
Network facts	ipaddress, dnspython, pysnmp	CIDR, DNS and authorized SNMP checks
Security graphs	networkx	Graph structures and algorithms
Planning	ortools	Constraint programming and optimization
Simulation	pybatfish	Batfish Python client
Hardening	ansible-core, ansible-runner, vendor collections	Controlled configuration change execution
Reports	jinja2, weasyprint; csv/json standard libraries	HTML, PDF, CSV and JSON output
Evidence	cryptography, hashlib standard library	Signed and hashed evidence
Observability	prometheus-client, opentelemetry-sdk	Metrics and traces
Testing	pytest, pytest-asyncio, hypothesis	Unit, integration and property tests
Quality	ruff, mypy	Linting and static types

5. Linux auditing - OpenSCAP and ComplianceAsCode

Tool	Purpose / decision
OpenSCAP / oscap CLI	Execute SCAP/XCCDF/OVAL checks and export compatible compliance results
ComplianceAsCode	Community security content, policy profiles, and supported remediation content
scap-security-guide	Packaged, distribution-appropriate SCAP content where available
SCAP Workbench (optional)	Graphical profile tailoring and content review
osquery	Structured system facts as supporting evidence; not standalone CIS certification
Native Linux commands	systemctl, auditd, sshd, PAM, sysctl, firewalld, nftables, UFW and file inspections
Custom CIS rule engine	Evaluate controls not covered by suitable version-specific SCAP content
Ubuntu Security Guide (optional)	Potential supplemental content where entitlement/compatibility permit; not a paid runtime dependency

Compatibility: Ubuntu 20.04 must remain supported by custom checks and appropriately validated version-specific content even if recent upstream ComplianceAsCode releases lack 20.04 support. Ubuntu 22.04 and RHEL 8/9 use compatible OpenSCAP content where available. CIS coverage must always be mapped to exact benchmark/version/profile; SCAP content is not automatically equivalent to every CIS Benchmark requirement.

6. Windows and OS-specific auditing

Platform	Read-only collection / evaluation	Approved hardening
Windows 11 Enterprise / standalone	PowerShell, WinRM, Registry, WMI/CIM, auditpol, secedit, LGPO, Policy Analyzer, osquery, custom CIS checks	PowerShell / WinRM, local policy or appropriate GPO changes
Windows Server 2019	Same Windows tooling plus server-specific security policies, accounts, services and logging	PowerShell, ansible.windows where compatible
Ubuntu 20.04 / 22.04 LTS	SSH, OpenSCAP/ComplianceAsCode where compatible, osquery, systemctl, PAM, auditd, sshd, UFW/nftables, sysctl	Ansible and validated Linux remediation templates
RHEL 8 / 9	SSH, OpenSCAP, SCAP Security Guide, osquery, auditd, SELinux, PAM, firewalld, sysctl	Ansible and RHEL-specific verified changes

Windows supplementary tool: Microsoft Security Compliance Toolkit (SCT) provides baseline comparison tools including LGPO and Policy Analyzer. Microsoft baselines and CIS Benchmarks are **not identical**; our rule mappings remain CIS-specific.

7. Firewall, router and switch integrations

Target	Collection / parsers	Remediation
Fortinet FortiGate	FortiOS APIs, SSH, firmware-specific parser	FortiOS API / compatible Ansible modules
Cisco firewall: ASA	ASA SSH/CLI, Netmiko, custom parser	ASA-specific verified automation
Cisco firewall: FTD	FMC/FTD management APIs, version-specific data	Appropriate FMC/FTD management API
Check Point	Management API, Gaia access / custom parser	Approved Management / Gaia workflows
Sophos	Firmware-specific XML or REST API, custom parsers	Sophos-specific API adapter
Palo Alto PAN-OS	REST/XML API and supported configuration parsing	PAN-OS API with required commit operations
Cisco routers	Netmiko/Scrapli, pyATS/Genie, TextFSM/TTP	cisco.ios or suitable platform-specific collections
Cisco managed switches	Netmiko/Scrapli, IOS/IOS-XE/NX-OS parser, pyATS/Genie	Platform-specific IOS/NX-OS automation

8. Networking support and laboratory tools

Tool	Why included
Nornir	Python-native network inventory and task dispatch
pyATS / Genie	Cisco platform parsing and network test automation
Nmap	Authorized discovery for inventory; strictly scope-limited
Wireshark	Troubleshoot protocols and network traffic
Scapy	Custom authorized network/protocol lab checks
TextFSM / TTP / ciscoconfparse2	CLI output and hierarchy parsing
Netmiko / Scrapli	SSH collection and verified network CLI commands
GNS3 / EVE-NG / Cisco CML where available	Virtual network test labs; vendor software may require licenses
VirtualBox / VMware	Windows and Linux VM integration tests
OpenSSH; Bruno / Postman	Connectivity and device/API diagnostics

9. Policy evaluation and CIS benchmark management

Tool / component	Role
Custom Python CIS Evaluation Engine	Primary deterministic source of truth for control checks
YAML / JSON / XML	Structured versioned rule packs
XCCDF + OVAL via OpenSCAP	Standards-compatible assessments where supported
Pydantic + JSON Schema	Strict policy pack validation and applicability checks
Open Policy Agent (OPA)	Declarative organization rules, approvals and authorization constraints
Git-based rule pack versioning	Traceable benchmark updates and changes
Custom mapping registry	CIS benchmark ID, release, profile, platform and evidence mapping
Custom applicability engine	PASS, FAIL, NOT_APPLICABLE, MANUAL_REVIEW, ERROR and NOT_ASSESSED

Rule safety: Do not evaluate untrusted YAML/JSON as arbitrary Python code. Use allowlisted, typed operations and fully tested control evaluators. Benchmark content must be legitimately sourced and its use/distribution rights respected.

10. Advanced innovation engines (no LLM required)

Custom engine	Primary tools	Responsibilities
Universal Configuration Normalizer	Pydantic, TTP, TextFSM, pyATS	Canonical structure across operating systems and network vendors
CIS Evaluation Engine	Python, YAML/JSON, OpenSCAP	Findings, evidence, rule applicability and versions
Cross-Layer Security Graph	NetworkX	Device and service relationships; cross-device risk graph
Root-Cause and Risk Analyzer	Graph traversals, deterministic correlation and scoring	Related misconfigurations and priority explanations
Minimal-Change Planner	OR-Tools CP-SAT	Optimize vetted remediation candidates under constraints
Conflict / Dependency Engine	NetworkX, explicit constraint rules	Sequencing, change conflicts and dependencies
Hybrid What-If Simulator	Batfish, Pybatfish, custom policy models	Before-after reachability and supported vendor behavior
Service-Safety / Blast-Radius Engine	Network path reasoning + validated probes	Required connectivity and predicted affected systems
Hardening Orchestrator	Ansible Runner, PowerShell, vendor APIs	Approval-gated deployment, state tracking, sequencing
Recovery Coordinator	Vendor checkpoints / backups / custom steps	Supported rollback, compensating changes, manual recovery
Dual Verification Engine	Re-audit, TCP/SSH/HTTP checks, Batfish	Compliance and required service post-checks
Drift & Evidence Engine	Supabase, Cryptography, Jinja2	Snapshots, trends, signed evidence and proof reports

11. Simulation, hardening and verification tools

Stage	Stack	Guardrails
Understand	NetworkX + normalized network/host facts	Only evidence-supported relationships
Plan	OR-Tools CP-SAT + vetted remediation templates	Respect hard constraints and required services
Simulate	Batfish + Pybatfish + custom semantics	Model coverage explicit; unmodeled behavior is NOT_VERIFIED
Harden	Ansible, Runner, PowerShell, Netmiko, vendor APIs	Read-only by default, explicit authorization, approval and backup
Verify	Fresh collection, rules, reachability/service probes	Do not substitute simulation for live verification
Prove	Signed artifacts, hashes, Jinja2/WeasyPrint reports	Before/after evidence, timestamps and approvals

Ansible **check/diff modes** are useful but not universally available; device-specific commit, backup and recovery behavior must be tested. Some changes are irreversible. For those, require controlled recovery and avoid promising automatic rollback.

12. Supabase, authentication and secrets

Service	Function
Supabase PostgreSQL	Organizations, assets, scans, controls, findings, approvals, change plans and audit history
Supabase Auth + MFA	Login, sessions, TOTP-based MFA and user management
Supabase RLS	Tenant isolation for authorized user-facing queries
Backend RBAC / authorization	Enforce permissions on API requests and privileged device changes
Supabase Storage	Private reports, encrypted configuration snapshots, backups and proof artifacts
Supabase Realtime	Dashboard updates, notifications and scan status
Supabase CLI / Studio	Local development, migrations and administration
Supabase JS / Python SDKs	Auth and storage integration points
OpenBao	SSH keys, privileged network-device passwords, API secrets
TLS / app-side encryption	Secure transport, encrypt sensitive artifacts before storing

Self-hosted Supabase is the deployment-independent option for sensitive environments. Supabase Cloud can be used with synthetic or sanitized data during development, subject to plan limits and organizational data handling requirements.

13. Data model and private storage buckets

Tables / entities	Purpose
organizations; organization_members; assets; asset_groups	Tenancy, roles, device inventory and groups
credential_references; benchmark_packs; policy_rules; policy_overrides	Vault pointers, benchmark definitions and organizational changes
audit_jobs; audit_results; findings; config_snapshots	Collection history, check results, misconfigurations and snapshots
security_relationships; risk_scenarios	Network/service connections and combined risk scenarios
remediation_plans; remediation_actions; approvals	Safe change planning, execution and authorization
safety_contracts; simulation_runs; verification_runs	Service constraints, simulations and live rechecks
audit_evidence; reports; audit_events; notifications	Evidence, exports, activity history and alerts

Private Storage buckets: `audit-evidence`, `config-snapshots`, `device-backups`, `generated-reports`. Encrypt sensitive content before upload and enforce access policies.

14. Reporting, evidence, observability

Tool	Role
Jinja2 + WeasyPrint	HTML and PDF findings/remediation reports
Python csv / json	Required CSV/JSON export formats
cryptography + hashlib (SHA-256)	Evidence signatures, integrity checks and artifact encryption
PostgreSQL append-controlled audit event model	Trace who scanned, approved, changed or verified
Prometheus + prometheus-client	Task and service metrics
Grafana	Operations dashboards
Loki	Central operational logs
OpenTelemetry	Cross-service tracing and diagnostic contexts

15. Deployment and testing toolchain

Category	Tools	What to test / operate
IDE and source	VS Code / Antigravity IDE; Git; GitHub	Code authoring and collaboration
Continuous integration	GitHub Actions	Automated tests and quality gates
Containers / ingress	Docker; Docker Compose; Linux; Nginx or Caddy	Service deployment and TLS reverse proxy
Supabase local stack	Supabase CLI; Supabase Studio	Local Auth, DB, Storage and migrations
API development	Bruno / Postman; Swagger UI; curl	API functionality and permissions
Backend tests	Pytest; pytest-asyncio; Hypothesis; Testcontainers	Rule fidelity, jobs and integrations
Frontend tests	Vitest; React Testing Library; Playwright	UI and end-to-end workflows
Static quality	Ruff; mypy; ESLint; TypeScript	Type and lint validation
Security scans	Bandit; Semgrep; Gitleaks; Trivy	Code, secrets and image/dependency issues
Dependency audits	pip-audit; npm audit	Known vulnerable packages
Web app security	OWASP ZAP	Security tests for app endpoints
OS labs	VirtualBox / VMware; Windows and Linux VM images	Platform-specific collector and hardening validation
Network labs	GNS3; EVE-NG; Cisco CML where available	Network configuration/simulation/automation validation
Packet diagnostics	Wireshark; Scapy	Connectivity and protocol troubleshooting

16. Custom development skills / engineering playbooks

Skill	What it standardizes
cis-benchmark-authoring	Versioned CIS rule schemas, mappings, review and test fixtures
openscap-compliance-content	OpenSCAP/ComplianceAsCode version and profile integration
windows-security-auditing	Windows policy collector and validation patterns
linux-security-auditing	Ubuntu/RHEL auditors and compatibility testing
vendor-adapter-development	Firewall, router and switch interface contracts
configuration-normalization	Canonical schema, parser coverage and error semantics
deterministic-risk-analysis	Cross-layer graph and explanation rules
remediation-optimization	OR-Tools models, feasible constraints and safety tests
network-simulation-testing	Batfish/custom model fixture comparison
safe-hardening-orchestration	Approvals, backups, prechecks and recovery
supabase-security	RLS, MFA, multi-tenant authorization and secrets isolation
proof-of-hardening	Evidence schema, signatures and audit report consistency
eighty-eight-feature-tracking	Feature ID, owner, test and verification status

Interpretation: These are proposed reusable coding-IDE skill/instruction files, not preinstalled tools and not runtime LLM components. Development may use AI-assisted coding if event rules allow it; the deployed auditor itself remains deterministic.

17. End-to-end system data flow

01 React/TypeScript GUI	Device and audit selection; findings, graph, simulation and approvals
↓	
02 Supabase Auth + FastAPI	Identity validation, RBAC, request-level checks
↓	
03 Celery + Redis + Nornir	Scheduled and parallel background audit jobs
↓	
04 Collection adapters	Authorized Windows/Linux/firewall/router/switch access
↓	
05 Normalize + CIS/SCAP checks	Versioned control evaluation and structured evidence
↓	
06 Graph + optimizer + simulator	Risk correlation, remediation planning and predicted change effects
↓	
07 Approval + hardening orchestration	Controlled supported configuration changes and recovery
↓	

08 Re-audit + service verification	Evaluate true post-change behavior
↓	
09 Supabase + OpenBao + reporting	Persist audit outcomes, protect secrets and distribute signed exports

18. All 88 planned feature entries - technical coverage

Feature group	Primary stack / implementation responsibilities
M01-M11 | All specified platforms	Windows/Linux collectors and five firewall-vendor + Cisco router/switch adapters
M12-M15 | Configuration collection	Nornir, Celery, WinRM, SSH, Netmiko, HTTPX, vendor APIs
M16-M19 | CIS engine, modular policies	Custom versioned CIS rules, OpenSCAP/ComplianceAsCode, JSON/YAML/XML
M20-M33 | Management, ACL, routing, switch, logging	Platform-specific deterministic evaluators with actual evidence
M34-M38 | GUI, scores, findings, reports	React, Supabase, Recharts, Jinja2, WeasyPrint, PDF/HTML/CSV/JSON
E01-E10 | Inventory, discovery, history	Supabase, authorized discovery, Celery schedules, snapshots, distributed workers
E11-E20 | Profiles, RBAC, exceptions, history	Rule versions, Supabase Auth/MFA/RLS, OpenBao, audit history
E21-E30 | Remediation and integrations	Ansible Runner, approvals, backups, vendor APIs, alerts and external integrations
I01-I05 | Cross-layer reasoning	NetworkX and deterministic configuration relationships
I06-I10 | Remediation optimization	OR-Tools CP-SAT, curated fixes and dependency constraints
I11-I15 | Simulation and safety	Batfish/Pybatfish, custom semantics, safety contracts, blast-radius analysis
I16-I20 | Verification and proof	Live re-audit, service tests, residual risk, signed evidence, benchmark upgrade analysis

19. Validation, constraints and implementation rules

- **Never remove a requirement silently.** All 38 mandatory entries and 50 extras stay in planned product scope; implementation status is tracked per feature.
- **No fake PASS states.** Expose PASS, FAIL, NOT_APPLICABLE, MANUAL_REVIEW, ERROR and NOT_ASSESSED as distinct outcomes. Report assessment coverage with compliance percentage.
- **Device-specific compatibility.** Vendors differ by appliance model, firmware and licensing. Maintain a collector/feature/command compatibility matrix.
- **Simulation is not proof.** Batfish and custom models predict supported behavior; actual post-change checks remain required.
- **No universal rollback guarantee.** Backup, approval, supported device recovery and manual escalation are parts of the workflow.
- **Benchmark rights matter.** CIS content requires correct license/permission. OpenSCAP and other security content do not automatically grant rights to bundle CIS Benchmarks.
- **Real infrastructure authorization.** Collectors and network discovery operate only against permitted devices/ranges, with least privilege and audit logs.
- **Security of the auditor itself.** Privileged secrets remain in OpenBao; sensitive configuration artifacts use private access controls and encryption.
- **Pin and verify dependencies.** Choose exact compatible library versions through integration testing; document unsupported features and recovery paths.

- **Keep runtime deterministic.** No LLM, AI/ML or paid auditing API dependency in the shipped decision engines.

20. Final stack summary

Frontend: React, TypeScript, Vite, Tailwind CSS, shadcn/ui, Recharts, Cytoscape.js, React Flow.

Backend and cloud: Python, FastAPI, Supabase PostgreSQL/Auth/Storage/Realtime, SQLAlchemy, Redis, Celery, Nornir, OpenBao.

Compliance and collection: OpenSCAP, ComplianceAsCode, osquery, Microsoft SCT, PowerShell, WinRM, Netmiko, Scrapli, pyATS/Genie, vendor APIs, custom CIS rule engine.

Innovations: NetworkX, OR-Tools CP-SAT, Batfish/Pybatfish, custom simulation, Ansible Runner, multi-vendor hardening, dual verification, cryptographic proof.

Engineering: Docker Compose, GitHub Actions, Pytest, Hypothesis, Playwright, GNS3/EVE-NG and observability/security scanners.

Final commitment 88-feature design scope retained: **M01-M38 + E01-E30 + I01-I20**. The complete original problem statement remains the foundation; additional intelligence, optimization, simulation and proof-of-hardening functionality form the product innovation layer.